

Lecture 22

Trefoil v4: Structs/Pattern Matching

Group Work

```
(define PI 3)
(struct circle r)
(struct rect l w)

(define (area s)
  (cond
    ((circle? s)
     (* PI (* (circle-r s)
              (circle-r s))))
    ((rect? s) (* (rect-l s)
                  (rect-w s)))))

(area (circle 4))
(area (rect 2 5))
```

```
type fn_info =
  {name: string
   ; params: string list
   ; body: expr}
type value =
  ...
  | Closure of fn_info * Env.t
  | Struct of string * value list

type expr =
  ...
  | StructConstructor of string * expr list
  | StructAccess of string * int * expr
  | StructPredicate of string * expr

type struct_binding =
  {name: string; fields: string list}
type binding =
  ...
  | Struct of struct_binding
```

Write out the closures that get added to the environment when each struct declaration is processed.

Group Work Answers

`circle → Closure({name: "circle"; params: ["x1"]; body: StructConstructor("circle", [Var "x1"])}}, env)`

`circle? → Closure({name: "circle?"; params: ["x"]; body: StructPredicate("circle", Var "x")}}, env)`

`circle-r → Closure({name: "circle-r"; params: ["x"]; body: StructAccess("circle", 0, Var "x")}}, env)`

`rect → Closure({name: "rect"; params: ["x1"; "x2"]; body: StructConstructor("rect", [Var "x"; Var "y"])}}, env)`

`rect? → Closure({name: "rect?"; params: ["x"]; body: StructPredicate("rect", Var "x")}}, env)`

`rect-l → Closure({name: "rect-l"; params: ["x"]; body: StructAccess("rect", 0, Var "x")}}, env)`

`rect-w → Closure({name: "rect-w"; params: ["x"]; body: StructAccess("rect", 1, Var "x")}}, env)`

Group Work

Add each of the following operators to Trefoil as a **Trefoil function** (if possible) or as a language **primitive** (if needed)

1. Boolean negation
 - `(not true) ⇒ false`
 - `(not false) ⇒ true`
2. Power
 - `(pow 2 3) ⇒ 8`
 - `(pow 4 2) ⇒ 16`
3. Greater Than
 - `(> 3 4) ⇒ false`
 - `(> 1 -1) ⇒ true`